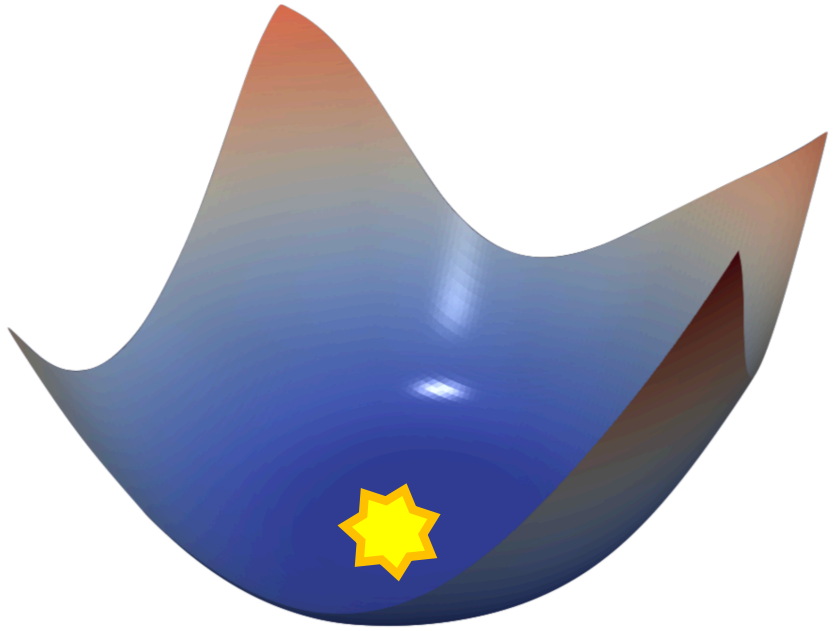
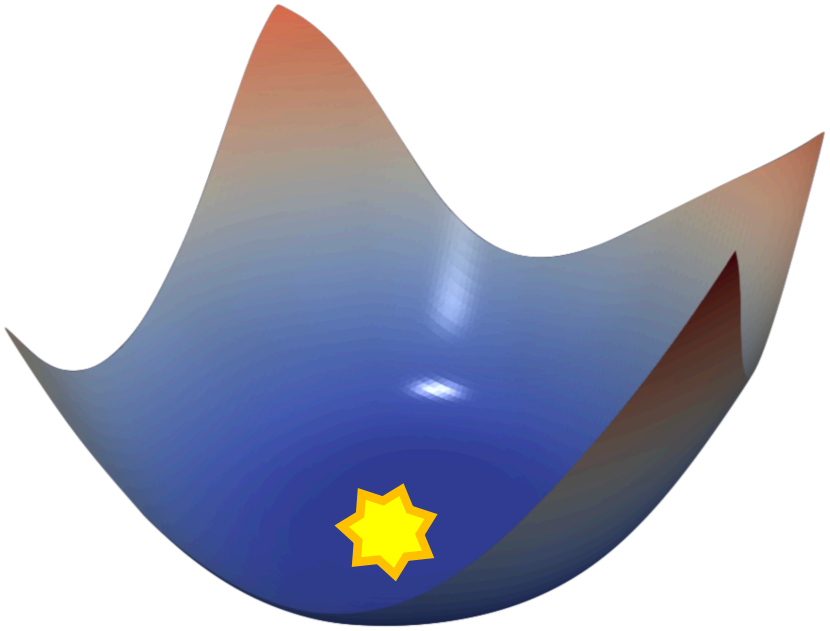
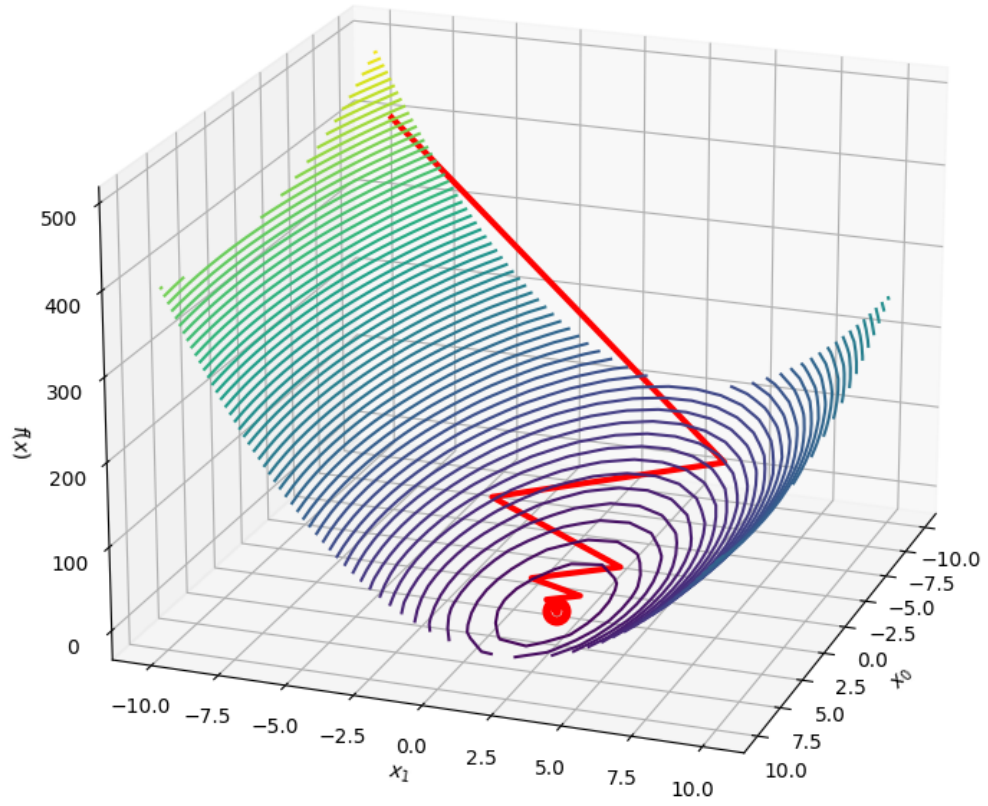
Optimizers

Why do we need an optimizer?

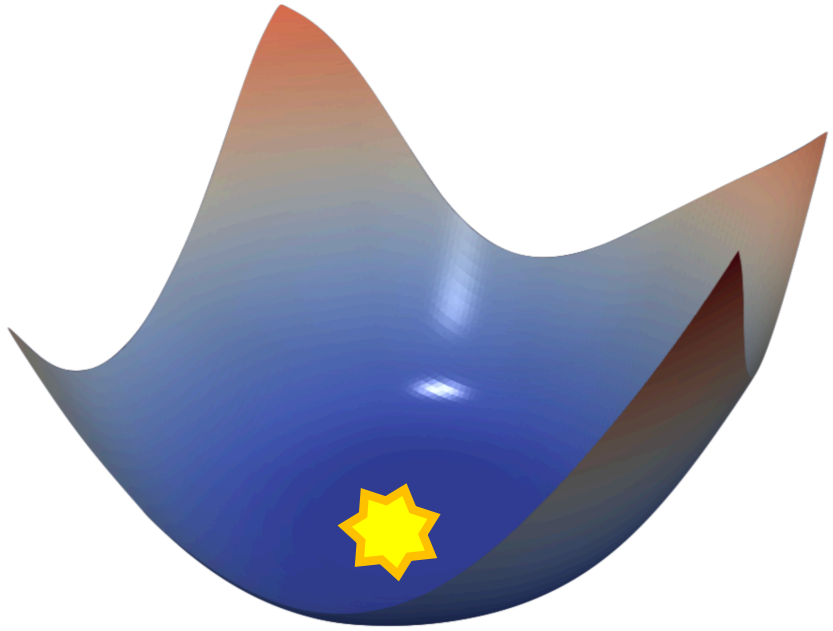


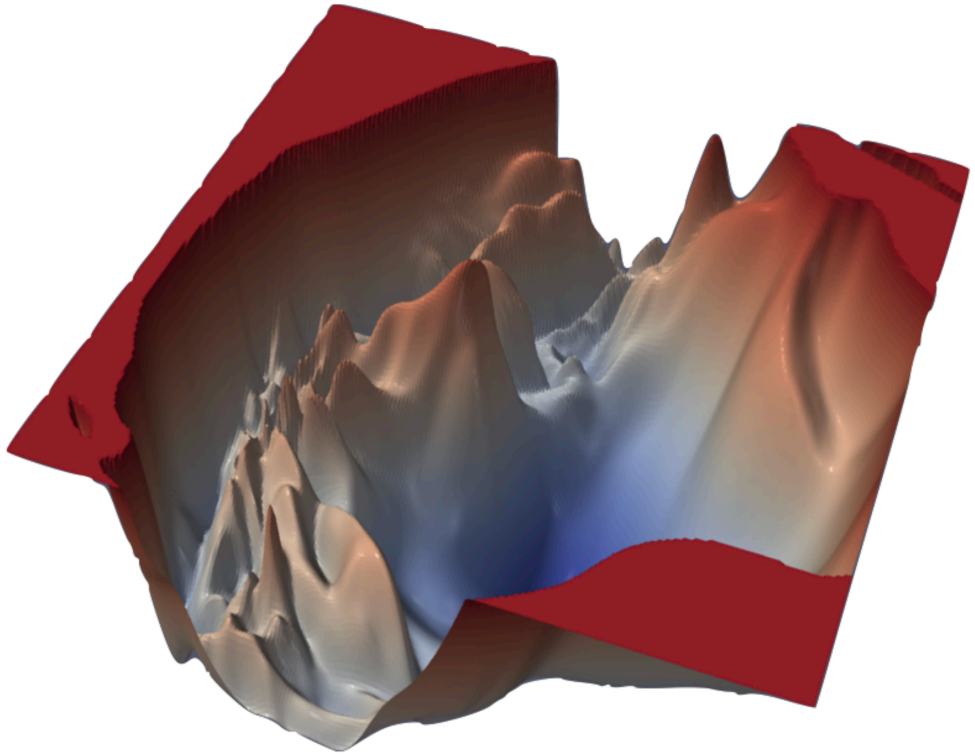
To solve a minimization problem





Sometimes the proble is very hard





Simple 2d problem: the Rosembrock function

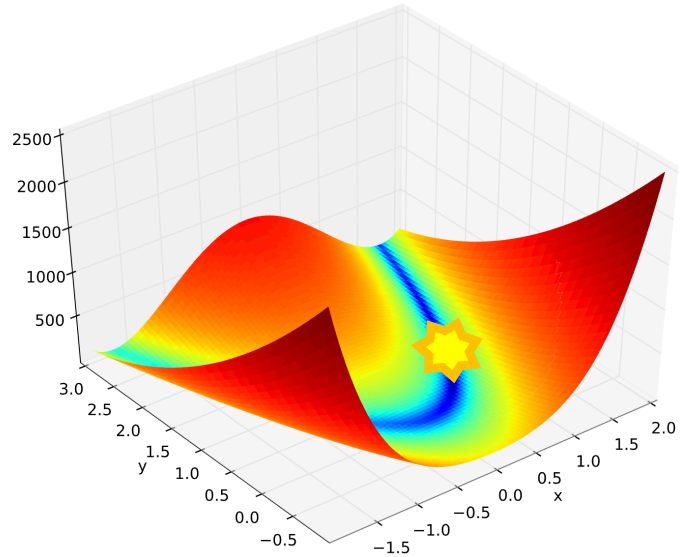
Let's consider a simple example:

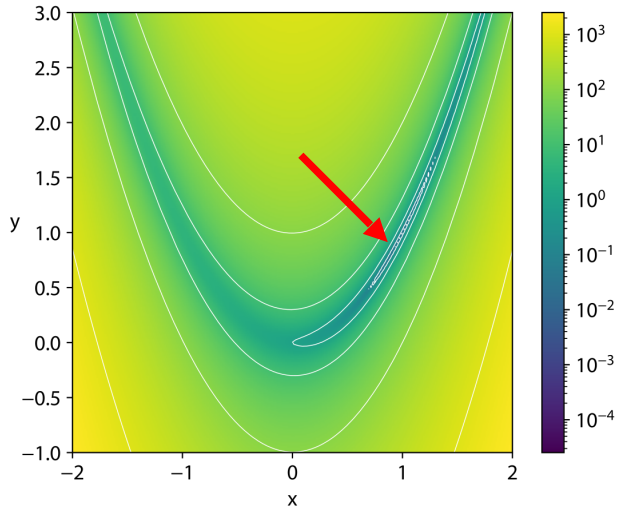
- a neural net with only 2 parameters $\boldsymbol{\theta}$
- a loss function $f : \mathcal{R}^2 \rightarrow \mathcal{R}$ mapping $\boldsymbol{\theta}$ into a scalar function

We can model this problem by looking at the minimum of the Rosenbrock function

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

where $\boldsymbol{\theta} = (x, y)$





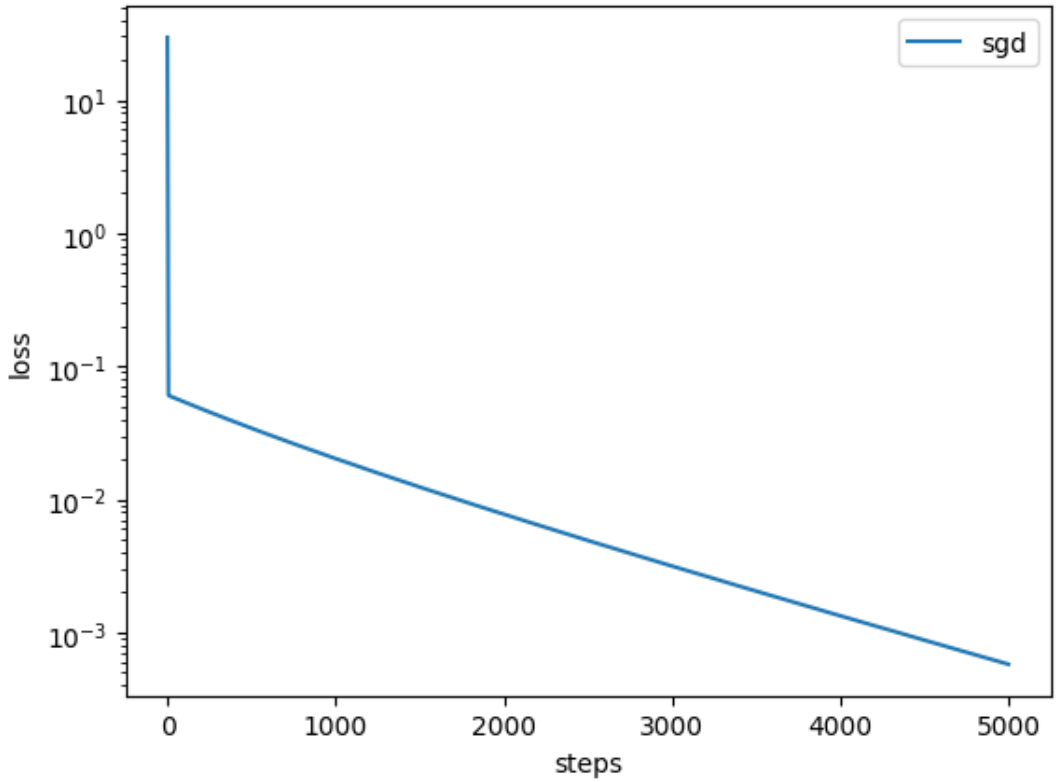
Stochastic Gradient Descent

for $t = 1, \ldots$ **do**
$\quad \boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta \nabla f_{\boldsymbol{\theta}_{t-1}}$
end for

```python
# Gradient Descend (2D)
lr = 1e-3
theta = np.random.random(size=(2, ))
step = 0

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    theta -= lr * grad_theta
    step += 1
```

Stochastic Gradient Descent

```python
# Gradient Descend (2D)
lr = 1e-3
theta = np.random.random(size=(2, ))
step = 0

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    theta -= lr * grad_theta
    step += 1
```



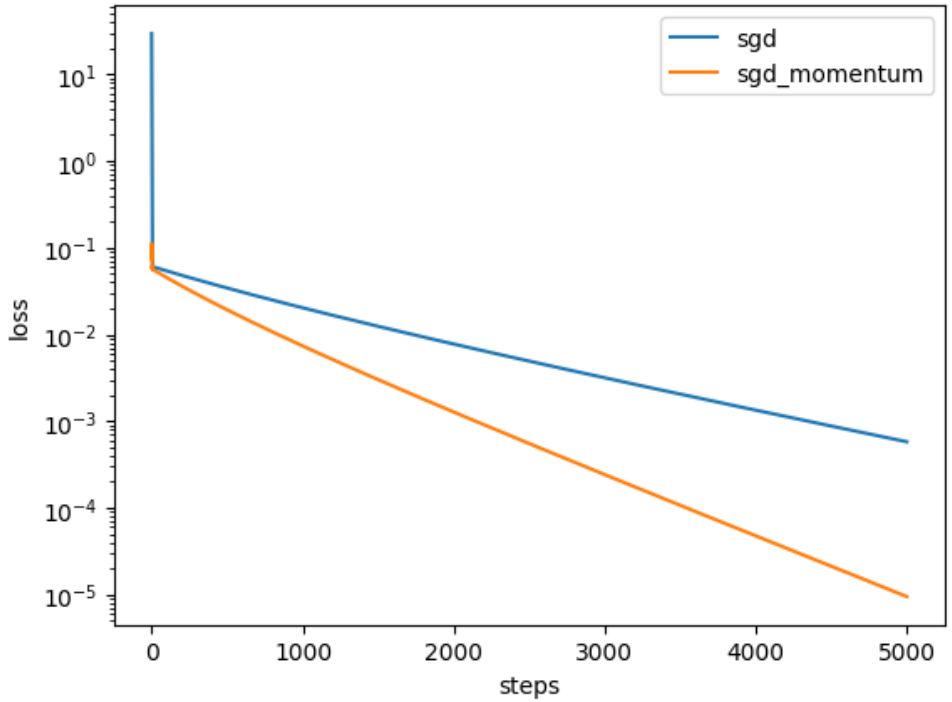
SGD with momentum

$$
\begin{aligned}
&\Delta\boldsymbol{\theta}_0 \leftarrow 0 \\
&\textbf{for } \mathrm{t} = 1, \ldots \textbf{ do} \\
&\quad \Delta\boldsymbol{\theta}_t \leftarrow \alpha\Delta\boldsymbol{\theta}_{t-1} - \eta\nabla f_{\boldsymbol{\theta}_{t-1}} \\
&\quad \boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} + \Delta\boldsymbol{\theta}_t \\
&\textbf{end for}
\end{aligned}
$$

```python
# adding momentum: remember the update dtheta at the last step,
# and find the new state as a linear combination between the
# last state and the last update
lr = 1e-3
theta, dtheta = np.random.random(size=(2, )), np.zeros_like(theta)
alpha = 0.5
step = 0

while step < steps:
    grad_theta = fun_grad(theta)  # gradients, shape (N, )
    dtheta = alpha * dtheta - lr * grad_theta # <- optimizer params, shape (N, )
    theta += dtheta
    step += 1
```

SGD with momentum

```python
# adding momentum: remember the update dtheta at the last step,
# and find the new state as a linear combination between the
# last state and the last update
lr = 1e-3
theta, dtheta = np.random.random(size=(2, )), np.zeros_like(theta)
alpha = 0.5
step = 0

while step < steps:
    grad_theta = fun_grad(theta)  # gradients, shape (N, )
    dtheta = alpha * dtheta - lr * grad_theta # <- optimizer params, shape (N, )
    theta += dtheta
    step += 1
```



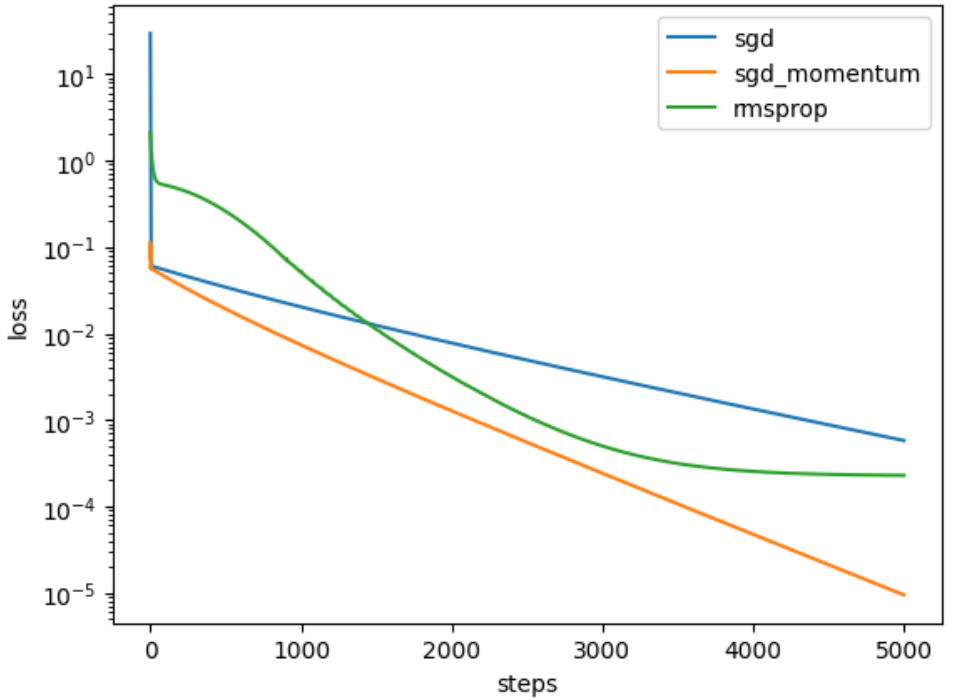
RMSprop: per-parameter learning rate

$$
\begin{aligned}
&\boldsymbol{v}_0 \leftarrow 0 \\
&\textbf{for } \mathrm{t} = 1, \ldots \textbf{ do} \\
&\quad \boldsymbol{v}_t \leftarrow \gamma \boldsymbol{v}_{t-1} + (1-\gamma)(\nabla f_{\boldsymbol{\theta}_{t-1}})^2 \\
&\quad \boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\boldsymbol{v}_t} + \epsilon} \nabla f_{\boldsymbol{\theta}_{t-1}} \\
&\textbf{end for}
\end{aligned}
$$

```python
# RMSProp: divide the learning rate by a factor that is the running
# average of the magnitudes of recent gradients for that weight
lr = 1e-3
theta, v = np.random.random(size=(2, )), np.zeros_like(theta)
gamma: float = 0.99
step = 0

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    v = gamma * v + (1 - gamma) * grad_theta ** 2 # <- optimizer params, shape (N, )

    theta -= lr * grad_theta / (np.sqrt(v) + eps)
    step += 1
```

RMSprop: per-parameter learning rate

```python
# RMSProp: divide the learning rate by a factor that is the running
# average of the magnitudes of recent gradients for that weight
lr = 1e-3
theta, v = np.random.random(size=(2, )), np.zeros_like(theta)
gamma: float = 0.99
step = 0

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    v = gamma * v + (1 - gamma) * grad_theta ** 2 # <- optimizer params, shape (N, )

    theta -= lr * grad_theta / (np.sqrt(v) + eps)
    step += 1
```



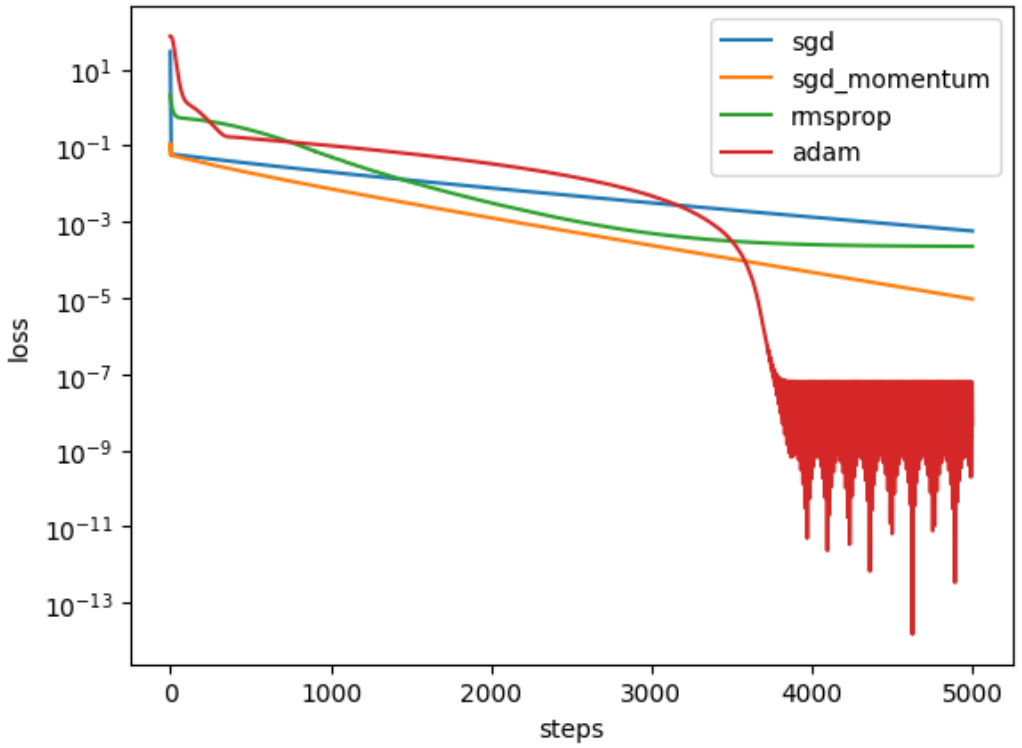
ADAM: momentum + running average

$$
\begin{aligned}
&\boldsymbol{v}_0 \leftarrow 0 \\
&\boldsymbol{m}_0 \leftarrow 0 \\
&\textbf{for } \mathrm{t} = 1, \ldots \textbf{ do} \\
&\quad \boldsymbol{m}_t \leftarrow \beta_m \boldsymbol{m}_{t-1} + (1-\beta_m)(\nabla f_{\boldsymbol{\theta}_{t-1}}) \\
&\quad \boldsymbol{v}_t \leftarrow \beta_v \boldsymbol{v}_{t-1} + (1-\beta_v)(\nabla f_{\boldsymbol{\theta}_{t-1}})^2 \\
&\quad \hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_m^t} \\
&\quad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_v^t} \\
&\quad \boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon} \\
&\textbf{end for}
\end{aligned}
$$

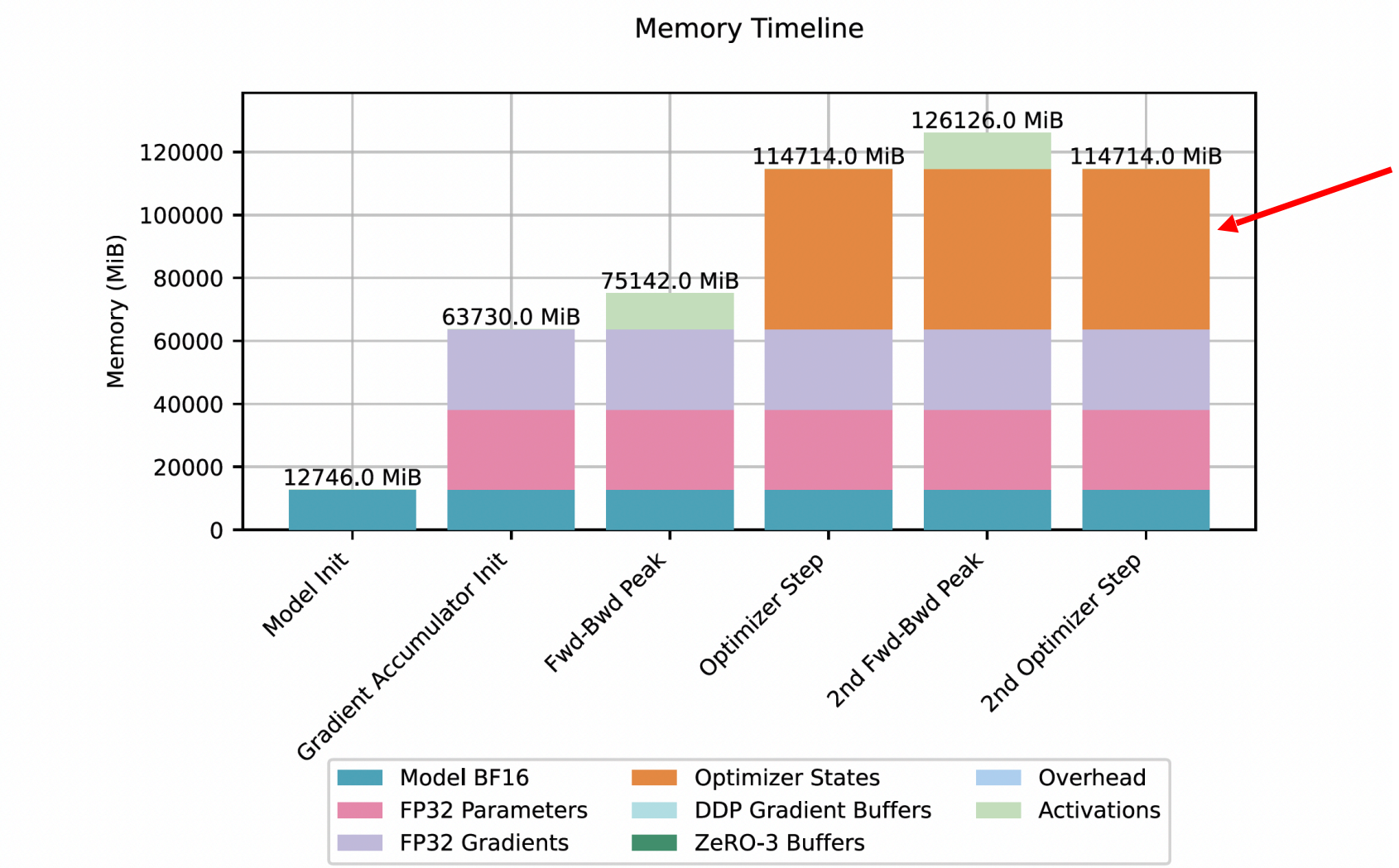
```python
# Adam: combine RMSprop with momentum.
lr = 2e-4
theta = np.random.random(size=(2, ))
v, m = np.zeros_like(theta), np.zeros_like(theta)
beta_v, beta_m = 0.9, 0.9
step = 0

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    m = beta_m * m + (1 - beta_m) * grad_theta  # <- optimizer params, shape (N, )
    v = beta_v * v + (1 - beta_v) * grad_theta ** 2 # <- optimizer params, shape (N, )

    v /= (1 - beta_v ** (step+1))
    m /= (1 - beta_m ** (step+1))

    theta -= lr * m / (np.sqrt(v) + eps)
    step += 1
```

ADAM: momentum + running average

```python
# Adam: combine RMSprop with momentum.
lr = 2e-4
theta = np.random.random(size=(2, ))
v, m = np.zeros_like(theta), np.zeros_like(theta)
beta_v, beta_m = 0.9, 0.9
step = 0

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    m = beta_m * m + (1 - beta_m) * grad_theta  # <- optimizer params, shape (N, )
    v = beta_v * v + (1 - beta_v) * grad_theta ** 2 # <- optimizer params, shape (N, )

    v /= (1 - beta_v ** (step+1))
    m /= (1 - beta_m ** (step+1))

    theta -= lr * m / (np.sqrt(v) + eps)
    step += 1
```



CUDA OOM: optmimizer matters



(Bonus) Newton's method: full 2nd order

for $t = 1, \ldots$ **do**
$\quad \boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - [H(f_{\boldsymbol{\theta}_{t-1}})]^{-1} J(f_{\boldsymbol{\theta}_{t-1}})$
end for

```python
def rosembrock_2d_hessian(theta:ArrayLike, a:float=1, b:float=100.) -> ArrayLike:
    x, y = theta
    hess_11 = 2 - 4 * b * (y - 3 * x ** 2)
    hess_12 = - 4 * b * x
    hess_21 = hess_12
    hess_22 = 2 * b
    return np.array([[hess_11, hess_12], [hess_21, hess_22]])
```

```python
# Newton: full 2nd order method
lr = 2e-4
theta = np.random.random(size=(2, ))
step = 0
steps = 10

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    hess_theta = rosembrock_2d_hessian(theta) # <- hessian, shape (N, N)

    theta -=  np.linalg.solve(hess_theta, grad_theta)
    step += 1
```

(Bonus) Newton's method: full 2nd order

```python
# Newton: full 2nd order method
lr = 2e-4
theta = np.random.random(size=(2, ))
step = 0
steps = 10

while step < steps:
    grad_theta = fun_grad(theta) # gradients, shape (N, )
    hess_theta = rosembrock_2d_hessian(theta) # <- hessian, shape (N, N)

    theta -=  np.linalg.solve(hess_theta, grad_theta)
    step += 1
```

```
step=0, theta=array([0.832, 0.212]), 23.1
step=1, theta=array([0.834, 0.696]), 0.0275
step=2, theta=array([1.   , 0.972]), 0.0754
step=3, theta=array([1., 1.]), 6.97e-09
step=4, theta=array([1., 1.]), 4.85e-15
step=5, theta=array([1., 1.]), 0.0
```